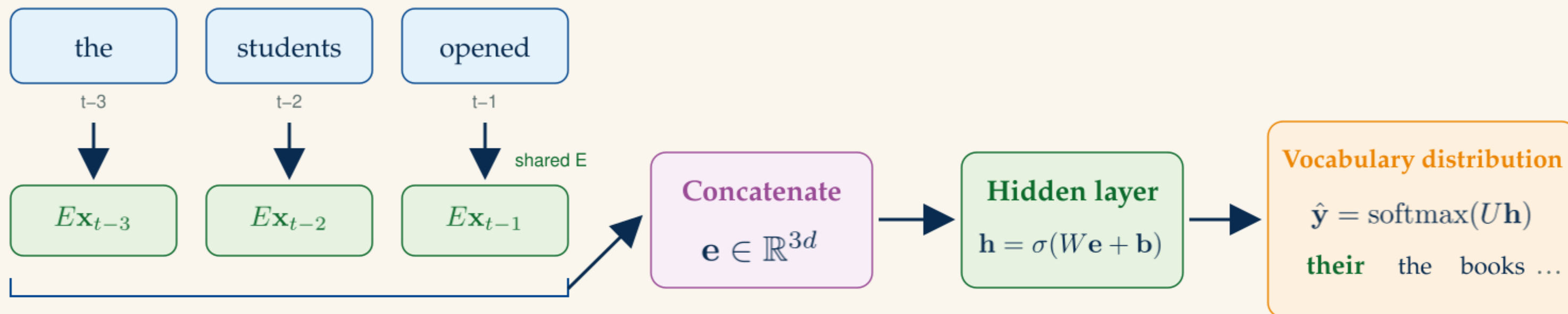


A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

FIXED CONTEXT WINDOW



$$\mathbf{e} = [E\mathbf{x}_{t-3}; E\mathbf{x}_{t-2}; E\mathbf{x}_{t-1}]$$

$$\mathbf{z} = U\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

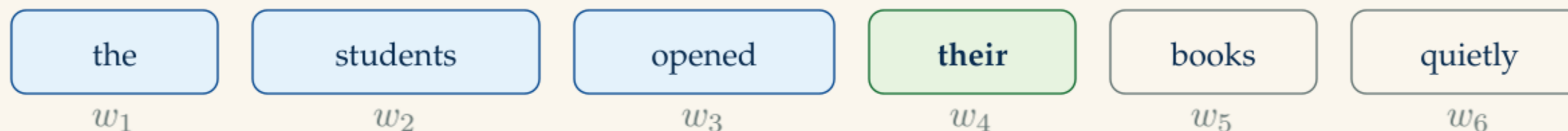
The output has one probability for every token in the vocabulary.

The context has order, but its length is fixed by the concatenated input.

Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

ONE SENTENCE PRODUCES SUCCESSIVE CONTEXT-TARGET PAIRS



At position t $\mathcal{L}_t = -\log \hat{y}_t[w_t]$ Negative log probability assigned to the observed next token.

TRAINING EXAMPLE 1



TRAINING EXAMPLE 2



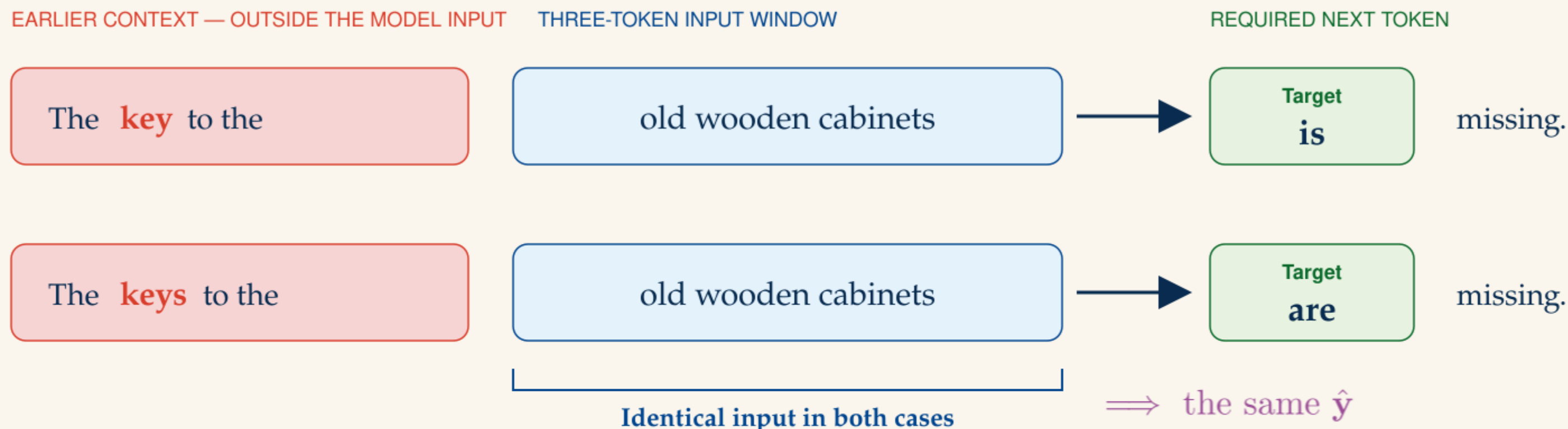
Training Corpus targets become mini-batched examples.

Inference Select or sample; shift the window; repeat.

Joint update Backpropagation updates the embedding matrix E and the network weights together.

The Model Still Sees a Fixed Context Window

Information before the chosen window is unavailable to the prediction.



$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-3}, w_{t-2}, w_{t-1})$$

The subject that determines agreement lies outside the window.

A larger window moves the boundary; it does not remove the boundary.

An RNN Processes an Ordered Sequence of Vectors

The recurrence can operate on sensor measurements, audio, video features, or text representations.

POSSIBLE SOURCES OF THE INPUT VECTOR

Sensor reading

One or more measurements at time t

$$x_t \in \mathbb{R}$$

Audio frame

Features from a short interval of sound

$$x_t \in \mathbb{R}^d$$

Video-frame features

Features extracted from the current frame

$$x_t \in \mathbb{R}^d$$

Word embedding

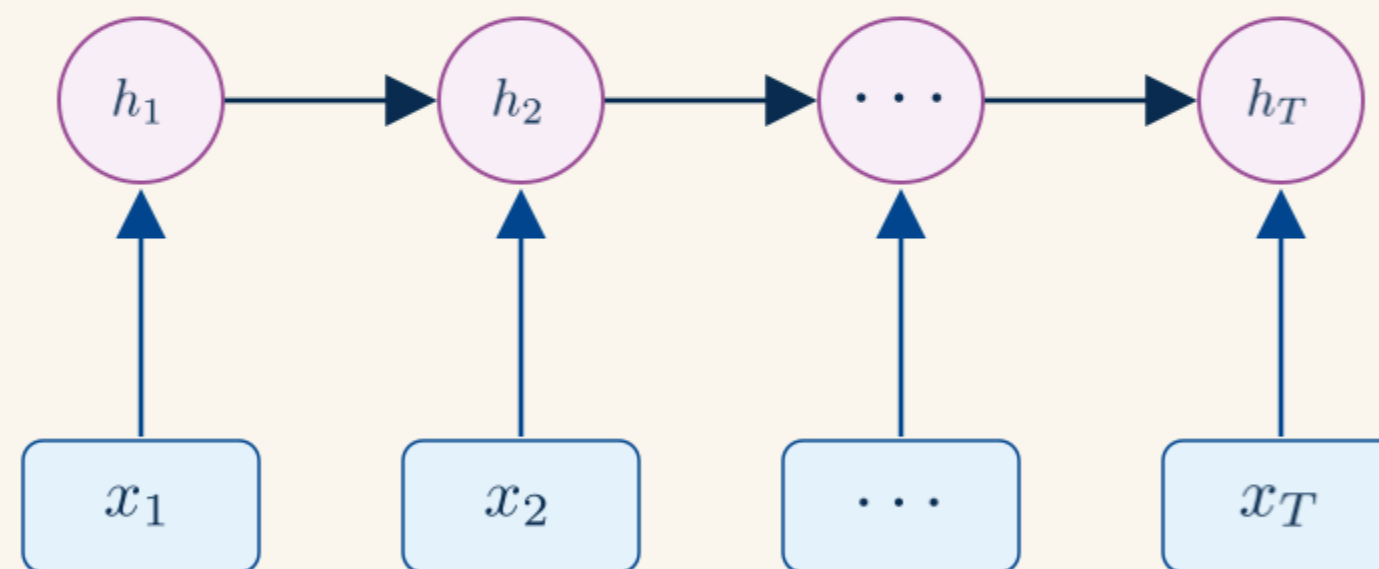
A vector representing the current token

$$e_t \in \mathbb{R}^d$$

THE SAME RECURRENT COMPUTATION

The RNN receives one vector at each step.

Only the vector values and their order enter the recurrence.



Variable-length sequence

The model sees an ordered sequence of vectors—not the original modality itself.

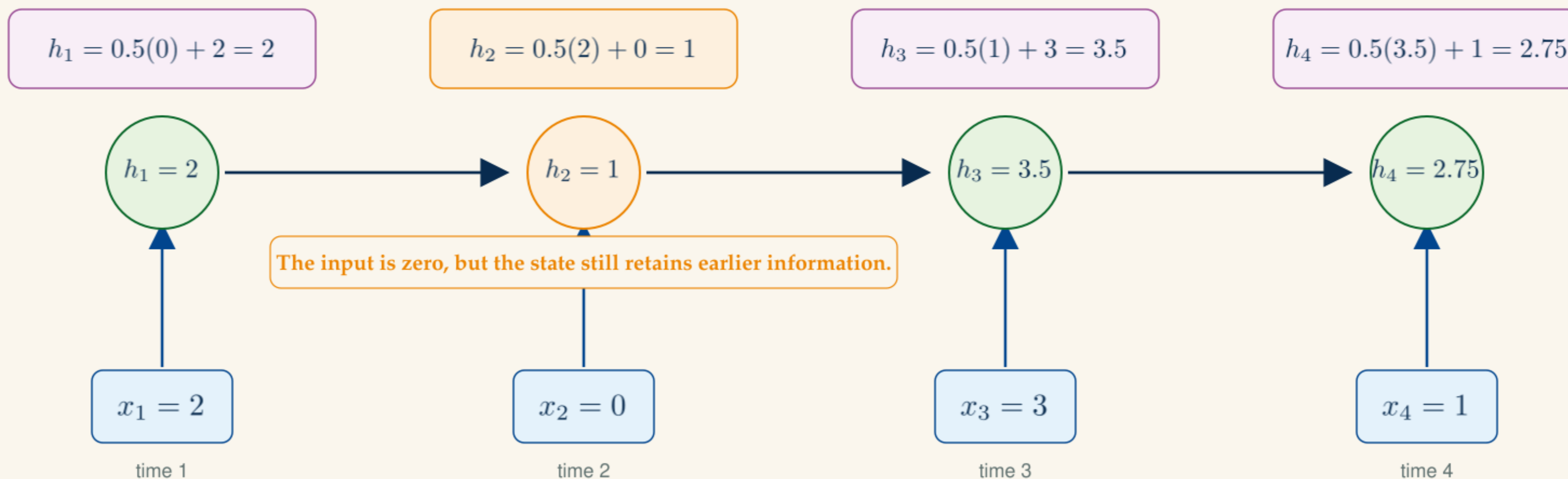
The Hidden State Represents the Signal So Far

Each observation updates—rather than replaces—the representation of the past.

A HAND-DESIGNED RECURRENCE FOR INTUITION

$$h_t = \frac{1}{2}h_{t-1} + x_t, \quad h_0 = 0$$

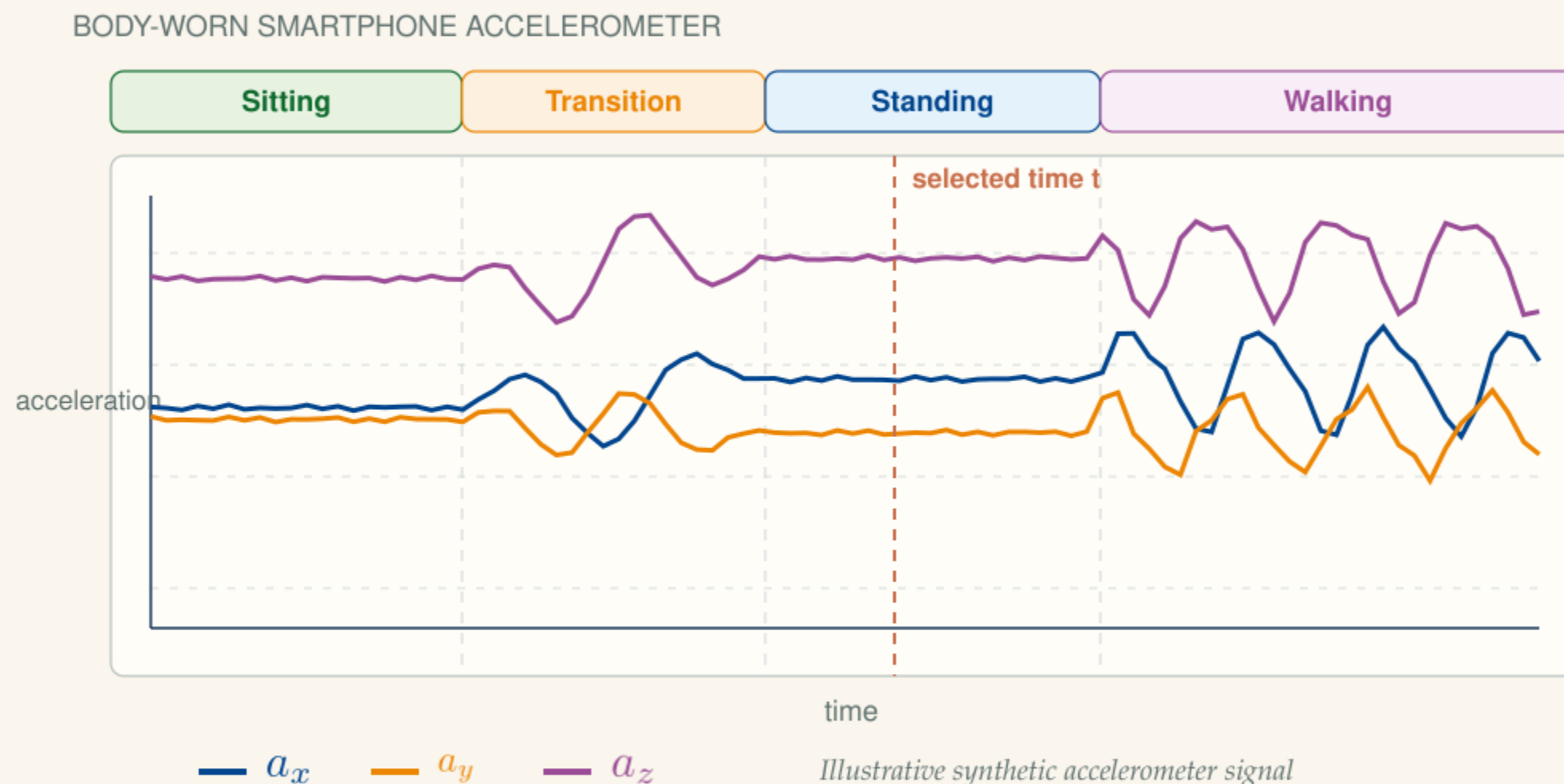
The new observation is combined with a retained part of the preceding state.



A vanilla RNN learns the update instead of fixing the coefficient by hand.

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.



ONE COLUMN AT TIME t

The sensor reports three numbers:

$$\mathbf{x}_t = (a_x(t), a_y(t), a_z(t))^T \in \mathbb{R}^3$$

one measurement vector

The complete recording is a matrix:

$$X = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_T] \in \mathbb{R}^{3 \times T}$$

columns are ordered in time

Possible labels at every time step:

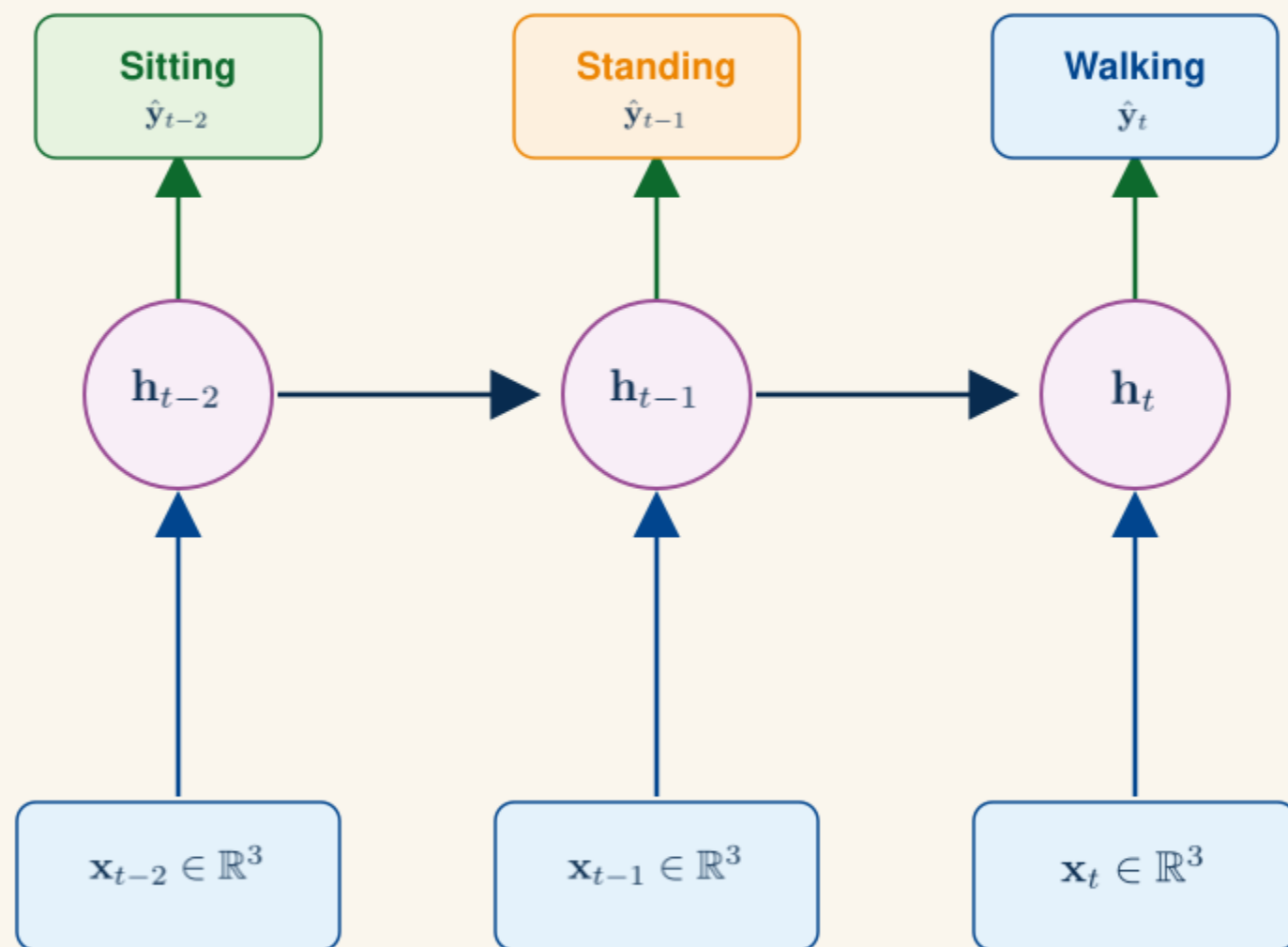
Sitting · Standing · Walking · Running · Dancing

Each vertical slice of the three-channel signal is one input vector.

Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



VECTOR DIMENSIONS IN THIS EXAMPLE

Update the running state

$$\mathbf{h}_t = g(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}_h)$$

Current evidence $W\mathbf{x}_t \in \mathbb{R}^m$

Carried history $U\mathbf{h}_{t-1} \in \mathbb{R}^m$

Predict the activity at time t

$$\hat{\mathbf{y}}_t = \text{softmax}(V\mathbf{h}_t + \mathbf{b}_y)$$

Input $\mathbf{x}_t \in \mathbb{R}^3$ State $\mathbf{h}_t \in \mathbb{R}^m$

Output $\hat{\mathbf{y}}_t \in \mathbb{R}^5$

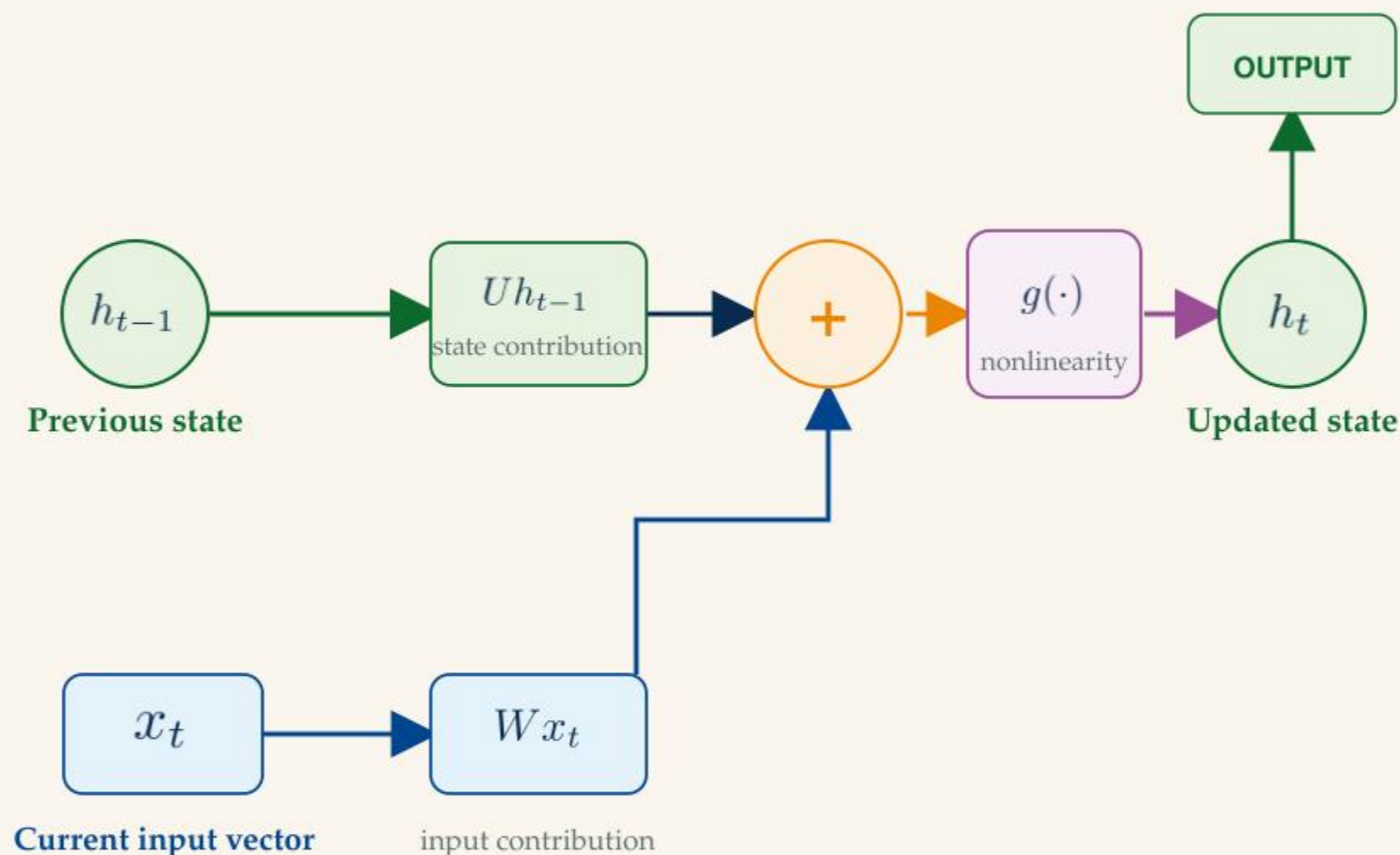
Why history helps

transitions and repeated gait patterns unfold across time

A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

ONE VANILLA-RNN STEP



THE LEARNED UPDATE

Update the hidden state

$$h_t = g(Uh_{t-1} + Wx_t + b_h)$$

Produce a task-specific output

$$\hat{y}_t = f(Vh_t + b_y)$$

Parameter roles

$$U : h \rightarrow h \quad W : x \rightarrow h \quad V : h \rightarrow y$$

The matrices are learned and reused at every time step.

Elman (1990): the simple recurrent network.

For text, the input vector is usually a learned word or token embedding.

Sequence Tasks Differ in Where Inputs and Outputs Appear

The task determines how many inputs are read and when predictions are made.

REFERENCE CASE

One-to-one



Image classification

SEQUENCE OUTPUT

One-to-many

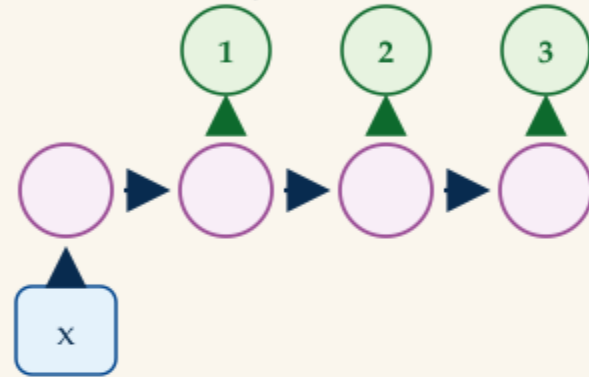
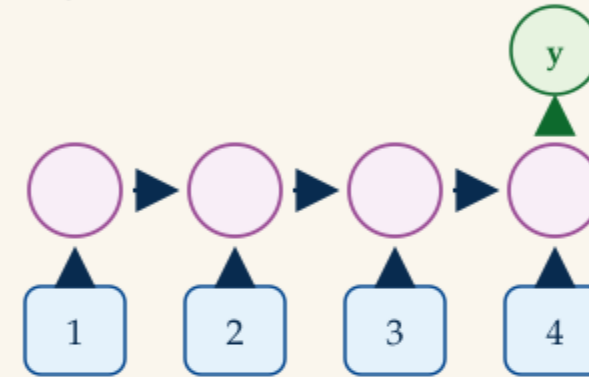


Image → caption

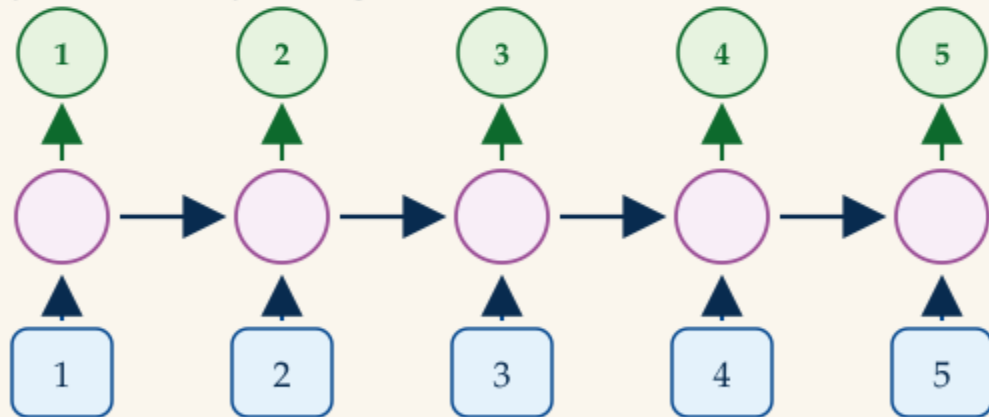
SEQUENCE INPUT

Many-to-one



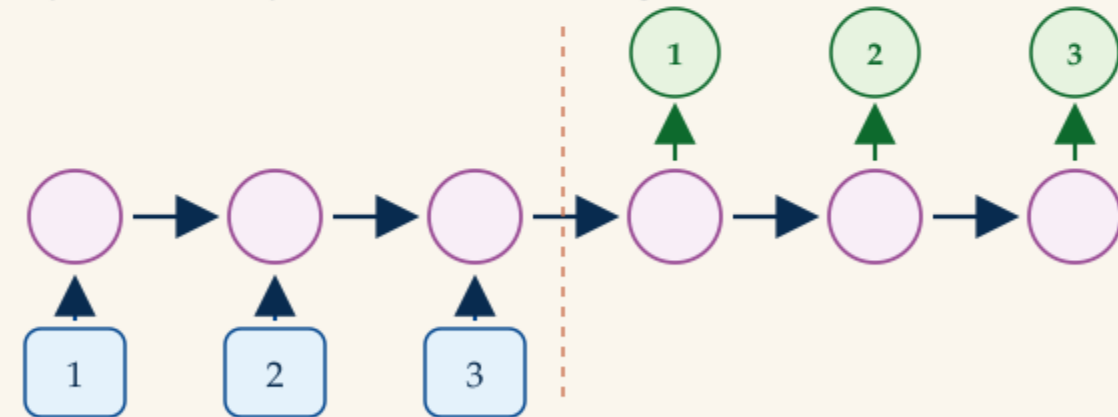
Review → sentiment

Many-to-many, aligned



Words → POS or entity tags

Many-to-many, different lengths



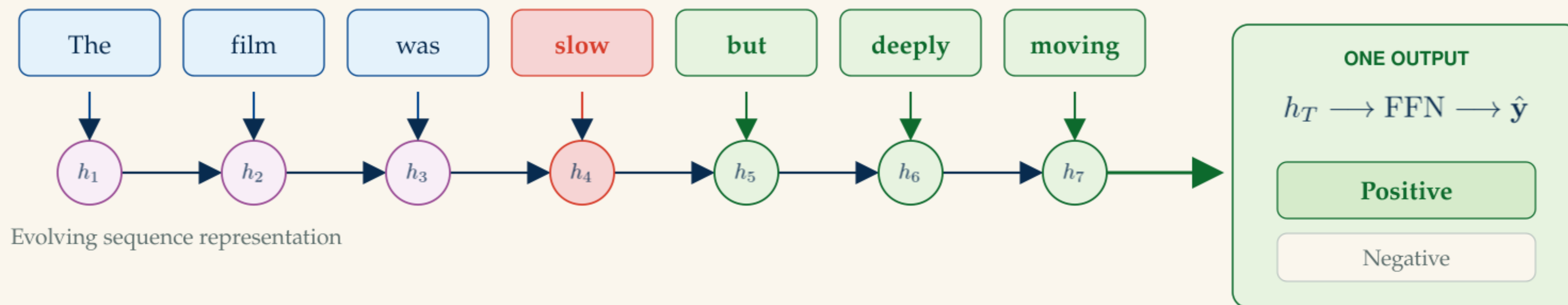
Sentence → translation

These are task structures—not categories exclusive to RNNs.

Many-to-One: Sentiment Analysis

A complete sequence is summarized for one task-level prediction.

THE SEQUENCE BUILDS ONE TASK-SPECIFIC REPRESENTATION



WHAT THE ARCHITECTURE ENABLES

- The prediction can wait until the complete sequence has been read.
- Later evidence can revise the representation built from earlier words.
- Training makes the final state useful for the downstream sentiment decision.

Baseline connection TF-IDF plus logistic regression ignores word order.

An RNN representation can depend on order, but this does not guarantee better performance.

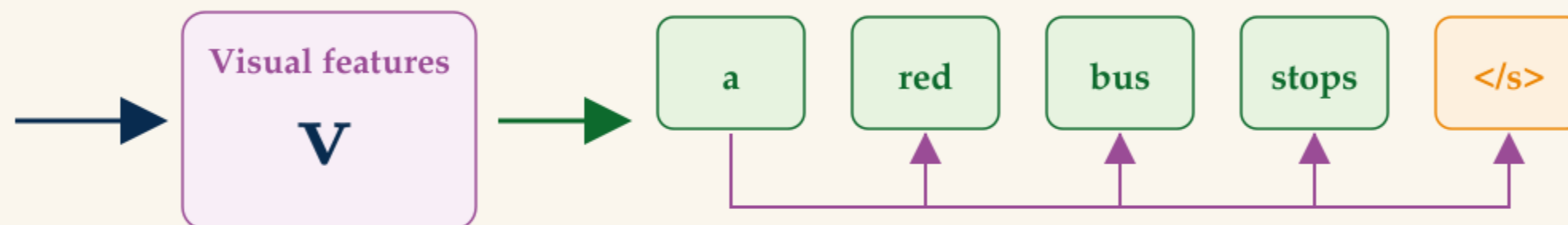
One-to-Many: Conditional Sequence Generation

A fixed input conditions a sequence produced one step at a time.

FIXED CONDITIONING INPUT



A fixed image



Each selected word becomes context for the next step.

Conditional generation

$$P(y_t \mid y_{<t}, \mathbf{v})$$

The sequence remains autoregressive.

OTHER ONE-TO-MANY APPLICATIONS

Music generation conditioned on a seed or style

Character generation conditioned on an initial prompt

HISTORICAL EXAMPLE

Show and Tell (2015)

CNN image encoder + recurrent caption decoder

Next: sequence inputs can produce aligned or different-length outputs.

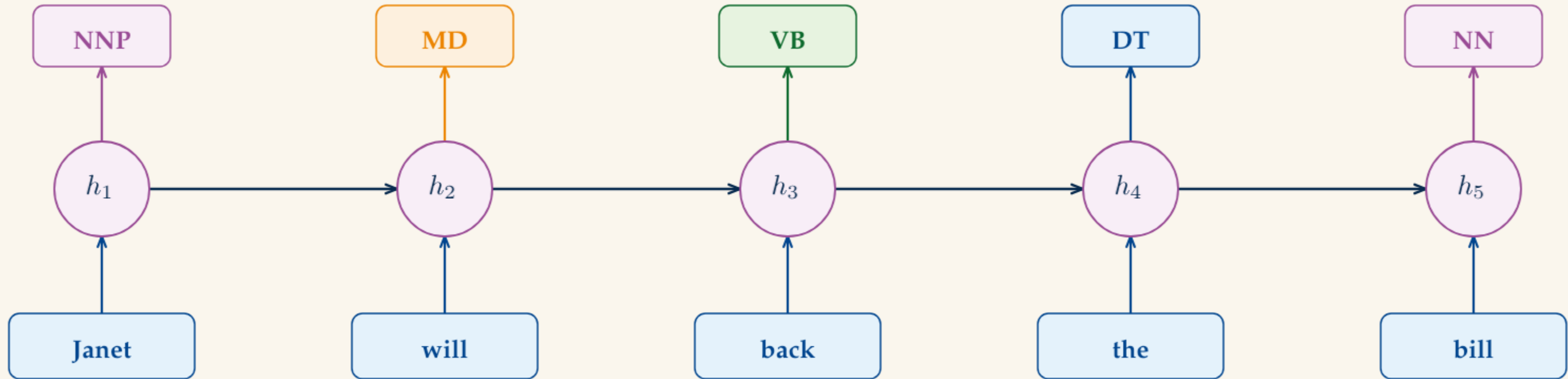
Many-to-Many: One Label for Every Input

Aligned sequence labelling produces one prediction at each input position.

ALIGNED PART-OF-SPEECH LABELLING

A TAG DISTRIBUTION AT EVERY POSITION

$$\hat{y}_t = \text{softmax}(Vh_t)$$



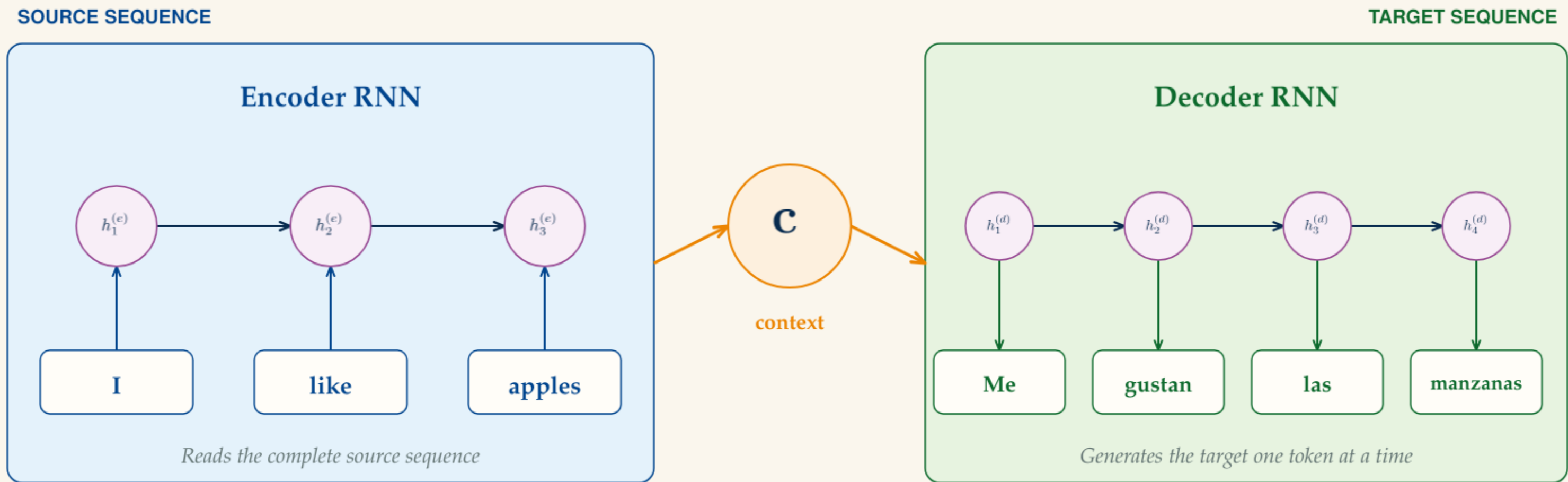
TAG KEY

NNP proper noun, singular	MD modal verb	VB verb, base form	DT determiner	NN noun, singular or mass
-------------------------------------	-------------------------	------------------------------	-------------------------	-------------------------------------

One input position produces one aligned label; the same parameters are reused throughout.

Outputs Need Not Align with Inputs

An input sequence can condition a separately generated output sequence.



$$P(y_{1:m} \mid x_{1:n}) = \prod_{t=1}^m P(y_t \mid y_{<t}, \mathbf{c})$$

The lengths may differ: m need not equal n .

Translation • Speech recognition • Summarization

The output at step t is not forced to align with the input at step t .

Sequence Tasks Differ in Where Inputs and Outputs Appear

The task determines how many inputs are read and when predictions are made.

REFERENCE CASE

One-to-one



Image classification

SEQUENCE OUTPUT

One-to-many

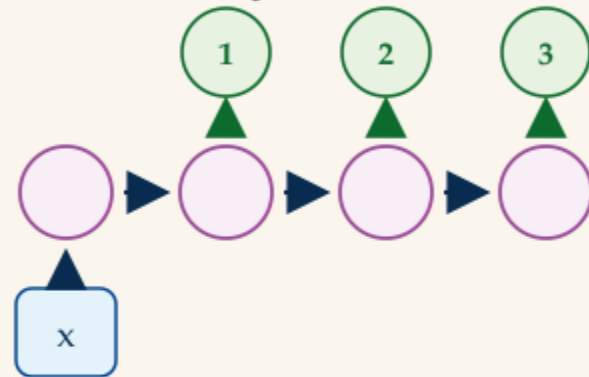
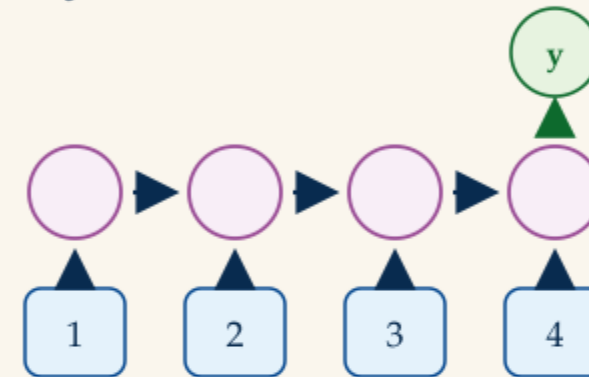


Image → caption

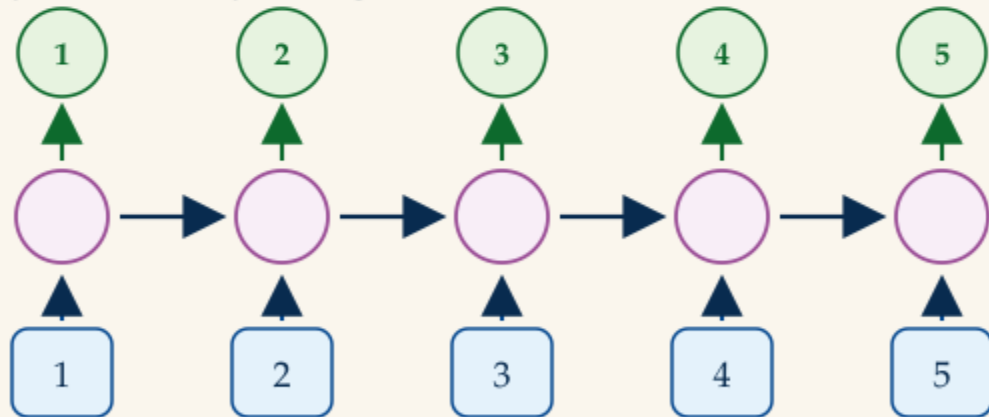
SEQUENCE INPUT

Many-to-one



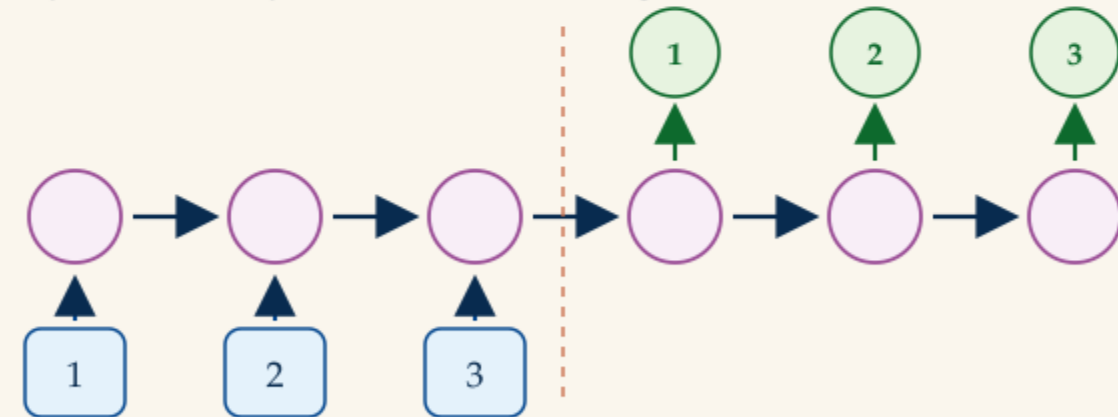
Review → sentiment

Many-to-many, aligned



Words → POS or entity tags

Many-to-many, different lengths



Sentence → translation

These are task structures—not categories exclusive to RNNs.

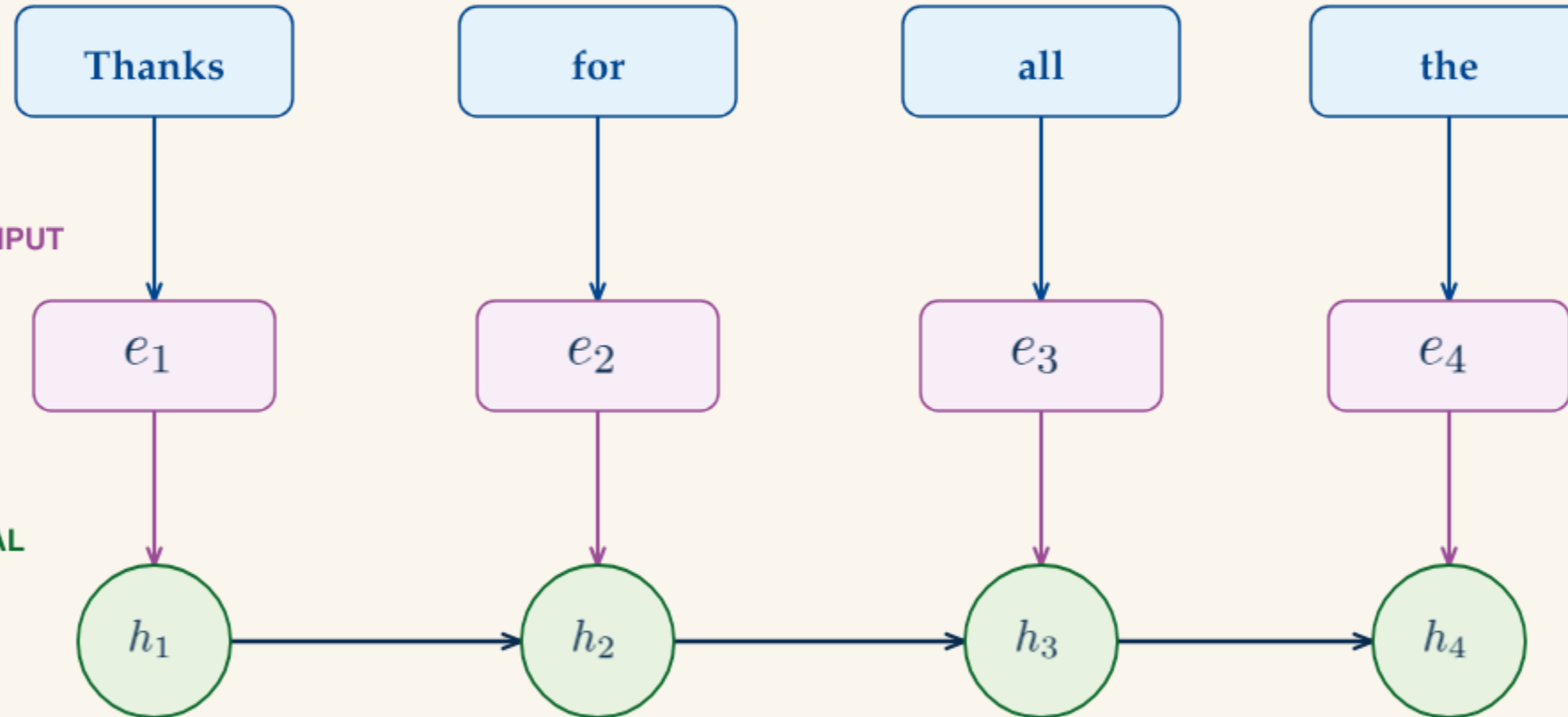
Words Become a Sequence of Input Vectors

An embedding lookup supplies the vector observed at each recurrent step.

THE SAME VECTOR-SEQUENCE INTERFACE, NOW FOR TEXT

TEXT-SPECIFIC FRONT END

DISCRETE
TOKENS



Embedding lookup

$$x_t \in \mathbb{R}^{|V|}$$

$$e_t = Ex_t \in \mathbb{R}^d$$

Recurrent update

$$h_t = g(Uh_{t-1} + We_t)$$

Where the vector comes from

Wearable: measured directly

Text: learned through E

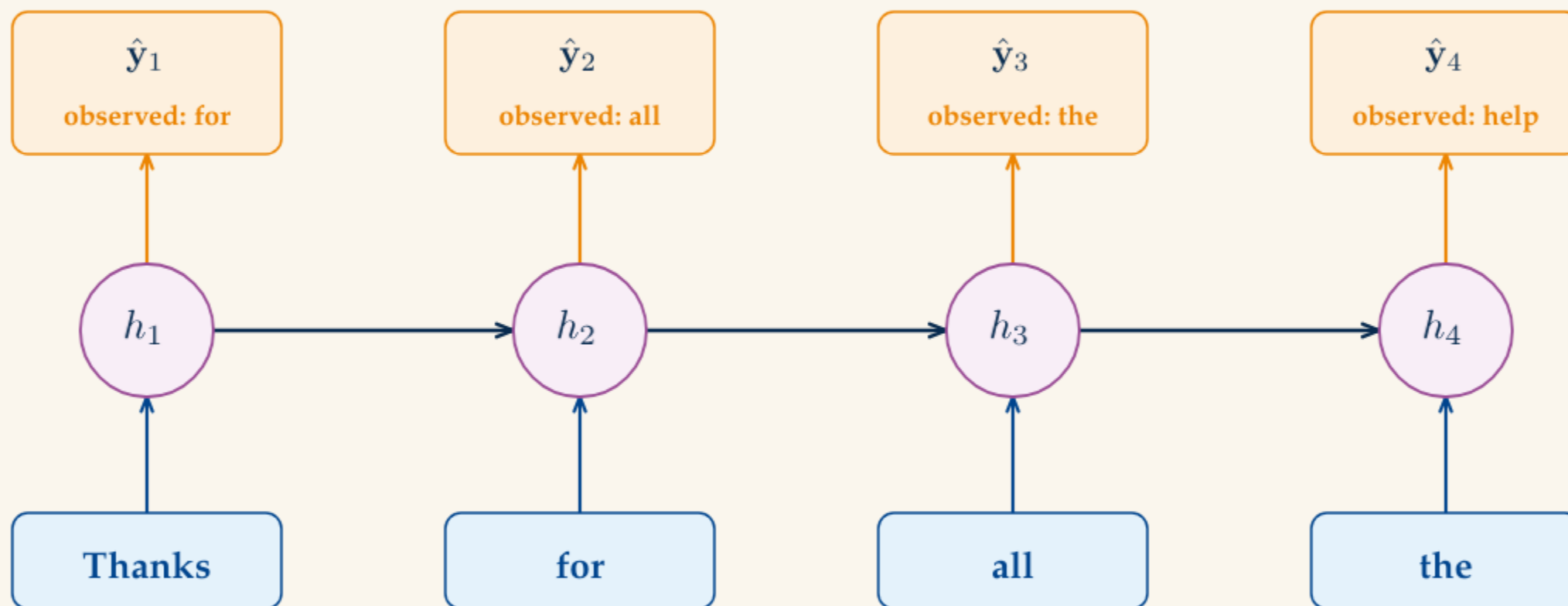
The recurrent computation is unchanged.

The embedding describes the current token; the hidden state describes the sequence so far.

Every Hidden State Predicts the Next Word

The recurrent state at position t defines a distribution over the vocabulary.

ONE NEXT-WORD DISTRIBUTION AT EVERY POSITION



Each orange box represents probabilities for every token in the vocabulary.

Vocabulary distribution

$$\hat{\mathbf{y}}_t = \text{softmax}(V h_t)$$

The k th component is the probability assigned to vocabulary word k .

$$P(w_{t+1} = k \mid w_{1:t}) = \hat{\mathbf{y}}_t[k]$$

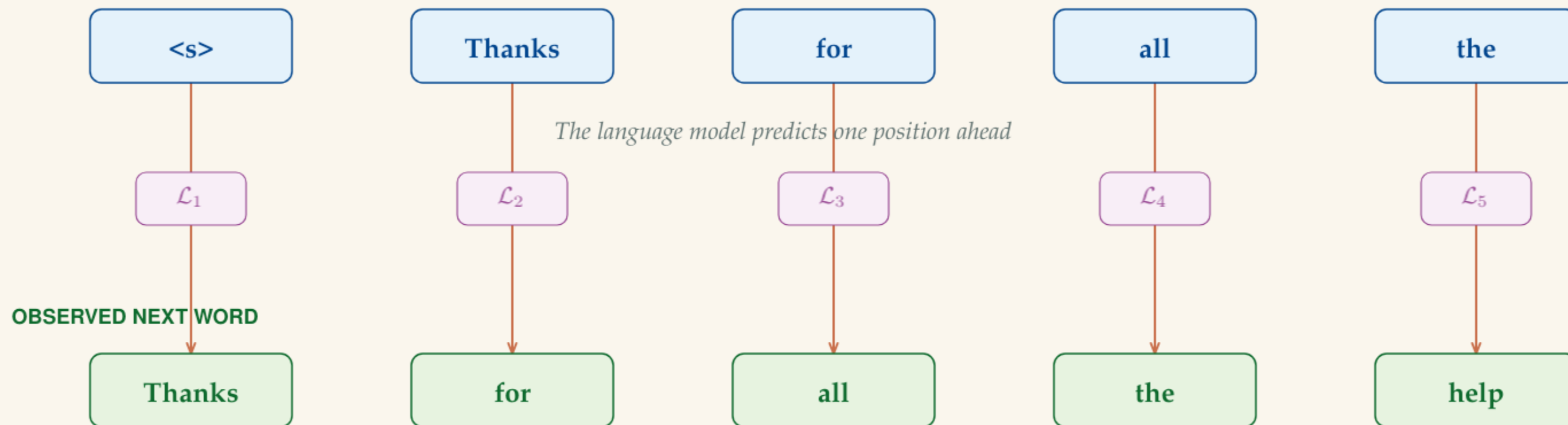
The output at t predicts position $t + 1$.

One sentence therefore produces several next-word predictions.

The Sentence Supplies Its Own Training Targets

Shifting the sequence by one position creates a target for every prediction.

MODEL INPUT



$$\mathcal{L}_t = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

No separately annotated labels are required.

The observed continuation is the correct class.

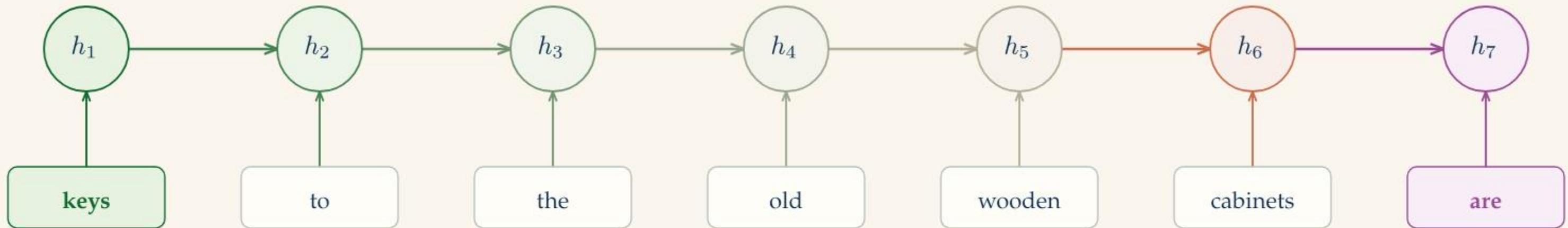
The state has access to earlier positions—but can their information survive?

Distant Information Must Survive Many State Updates

A recurrent path provides access to the past, but it does not guarantee reliable memory.

SUBJECT INFORMATION ENTERS HERE

AGREEMENT IS PREDICTED HERE



The path exists, but useful information must survive every intervening update.

During training, many gradient factors multiply

$$\frac{\partial h_T}{\partial h_t} = \prod_{j=t+1}^T \frac{\partial h_j}{\partial h_{j-1}}$$

The product may shrink toward zero or grow rapidly.

The vanilla RNN has no explicit memory controls

Earlier information may be overwritten.

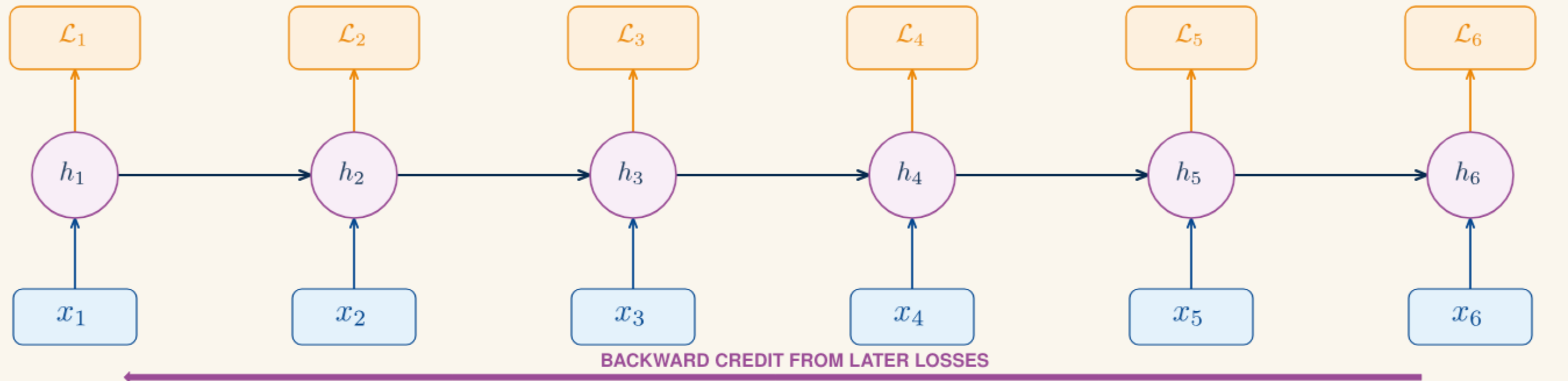
There is no learned keep / replace decision.

The same repeated path carries information forward and credit backward.

BPTT Sends Credit Back Through the Unrolled Network

Truncated BPTT carries state forward but detaches the gradient at chunk boundaries.

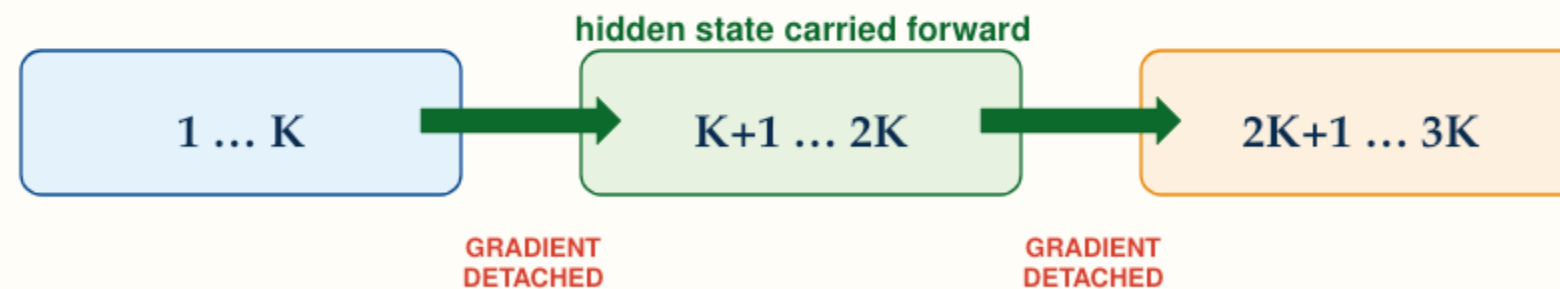
FULL BACKPROPAGATION THROUGH TIME



TRUNCATED BPTT

The sequence is processed in chunks of K steps. The last state becomes the first state of the next chunk.

State continues; gradient stops.



Shorter backward paths save computation—but cannot teach dependencies across a cut.

Repeated Jacobians Can Make Gradients Vanish or Explode

The backward signal is repeatedly transformed as it travels toward earlier time steps.

$$\frac{\partial h_T}{\partial h_t} = \prod_{j=t+1}^T \frac{\partial h_j}{\partial h_{j-1}}$$

VANISHING GRADIENT

Local factors repeatedly shrink the signal

$$0.5^{10} \approx 0.001$$



Early time steps receive almost no learning signal from later losses.

Long-range dependencies are difficult to learn.

EXPLODING GRADIENT

Local factors repeatedly amplify the signal

$$1.5^{10} \approx 57.7$$



Gradient norms become very large, producing unstable parameter updates.

Training may oscillate or diverge.

Both arise from the same multiplication chain; they require different responses.

The Two Gradient Problems Need Different Responses

Clipping bounds unstable updates; gated state paths support longer-range credit assignment.

EXPLODING GRADIENTS

Clip the gradient norm

$$g \leftarrow g \min\left(1, \frac{\tau}{\|g\|_2}\right)$$



oversized gradient $\rightarrow$ bounded update

Direction is preserved; the norm is capped at τ . Memory is unchanged.

VANISHING GRADIENTS

Build an additive carry path



A near-identity route can retain state and credit.

GRUs and LSTMs learn when to preserve or replace information.

TBPTT is a computational compromise—not a cure for vanishing gradients.

Next: replace a fixed carry weight with a learned gate.